```
# WEEK-2 PROGRAMMING ASSIGNMENT-1
# Discount Calculator for TechMart
# TechMart is running a 10% discount sale on all products. When a customer purchases an item, the cashier enters the original price into the billing system. The system calcula
#
# Write a Python program that:
# (a) Accepts the original price of an item as input.
# (b) Converts the entered value into a number.
# (c) Calculates the final price after applying a 10% discount.
# (d) Prints the discounted price.

original_price = int(input())
discounted_price = original_price - (original_price * 0.1)
print(discounted_price)
```

```
 500
450.0
```

```
# WEEK-2 PROGRAMMING ASSIGNMENT-2
# Library Book Filter
# A school library maintains a list of the number of pages in newly arrived books. To help younger students, the librarian wants to identify books that have more than 300 pag

# Write a Python program that:
# (a) Accepts a space-separated list of integers representing the number of pages in each book.
# (b) Identifies all books with more than 300 pages.
# (c) Prints the page counts of these books, separated by spaces.

lst = list(map(int , input().split()))
for i in lst:
    if i >300:
        print(i)
```

```
 301 201 350 620
301
350
620
```

```
#WEEK-2 PROGRAMMING ASSIGNMENT-3

# Even Number Counter
# A teacher is conducting a classroom activity on number patterns. Students are asked to enter a list of numbers, and the computer counts how many of them are even.

# Write a Python program that:
# (a) Accepts a space-separated list of integers.
# (b) Counts how many numbers are even.
# (c) Prints the count.

lst = list(map(int ,input().split()))
count = 0
for i in lst:
    if i%2 == 0:
        count = count + 1
print(count)
```

```
 8 9 10 11
2
```

```
# WEEK-3 PROGRAMMING_ASSIGNMENT-1
# During a school sports meet, the finishing times (in seconds) of runners are recorded. A lower time indicates a faster runner.

# (a) Accept a space-separated list of positive real numbers representing finishing times.
# (b) Find the minimum finishing time.
# (c) Print the fastest finishing time.

lst = list(map(float , input().split()))
print (min(lst))
```

```
 18.2 17.9 18.0
17.9
```

```
# WEEK-3 PROGRAMMING_ASSIGNMENT-2
# A weather station records daily temperatures as decimal values.

# (a) Accept a space-separated sequence of real numbers.
# (b) Convert each temperature to its integer part.
# (c) Print the sum of all converted integer values.

lst = list (map(float , input().split()))
res = 0
for i in lst:
    res += int(i)
print(res)
```

```
 40.0 39.9
79
```

```
# WEEK-3 PROGRAMMING ASSIGNMENT-3
# A courier company records package weights (in kg).

# (a) Accept a space-separated list of positive real numbers.
# (b) Compute the average package weight.
# (c) Count how many package weights are greater than the average.

lst = list(map(float , input().split()))
avg = sum(lst)/len(lst)
count = 0
for i in lst:
    if i<avg:
        count+=1
print(count)
```

```
 2.2 2.4 2.6 2.8
2
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

```
# WEEK-4 PROGRAMMING ASSIGNMENT-1
# Modern weather stations continuously record temperature readings throughout the day. At the end of the day, the system needs to determine how much the temperature varied.

# Write a Python program that:
# Accepts space-separated integers representing temperature readings.
# Finds the highest and lowest temperature recorded.
# Calculates and prints the temperature range, defined as the difference between the highest and lowest readings.
# If there is only one reading, print 0.

lst = list(map(int , input().split()))
print( max(lst)-min(lst) if lst !=1 else 0)  #returns 0 if lst have one reading or value
```

```
 12 18 5 21 14
16
```

```
# WEEK-4 PROGRAMMING ASSIGNMENT-2
# A cybersecurity application generates numeric authentication codes. Before using a code, the software checks whether it is a prime number.

# Write a Python program that:
# Accepts a positive integer n.
# Prints True if n is a prime number; otherwise, prints False.

n = int(input())
isPrime = True
if n < 2:
    isPrime = False
for i in range(2, n//2+1):
    if n%i == 0:
        isPrime = False
print(isPrime)
```

```
 29
True
```

```
#WEEK-4 PROGRAMMING ASSIGNMENT-3
# A shopping platform records the product IDs viewed by a customer during a browsing session. The system needs to determine whether the customer viewed any product more than

# Write a Python program that:
# Accepts space-separated integers representing product IDs.
# Prints True if at least one product ID appears more than once.
# Otherwise, prints False.

lst = list(map(int, input().split()))
isDuplicate = False
for i in range(len(lst)):
    for j in range(i+1,len(lst)):
       if lst[i]==lst[j]:
         isDuplicate=True
         break
print(isDuplicate)
```

```
 1 4 8 9
False
```

```
Start coding or generate with AI.
```

```
#WEEK-5 PROGRAMMING ASSIGNMENT-1
# A data analyst receives a sequence of integers representing values recorded by a sensor. Some values may occur multiple times.

# Write a function that:
# (a) Accepts a space-separated sequence of integers.
# (b) Returns the number that occurs most frequently.
# (c) If two or more numbers have the same highest frequency, returns the smallest number.

lst = list(map(int, input().split()))
freq={}

for nums in lst:
    if nums in freq:
        freq[nums]+=1
    else:
        freq[nums]=1
high = 0
res = None

for nums in freq:
    if freq[nums] > high:
        high = freq[nums]
        res = nums
    elif freq[nums] == high and nums < res:
        res = nums
print(res)
```

```
5 2 5 2 4 3
2
```

```
# WEEK-5 PROGRAMMING ASSIGNMENT-2
# A system receives a sequence of integers. Some numbers may appear more than once.

# Write a function that:
# (a) Accepts a space-separated sequence of integers.
# (b) Returns all numbers that occur more than once.
# (c) The numbers should be returned in the order of their first appearance.
# (d) If no number is repeated, returns -1.

def duplicate():
    lst = list(map(int, input().split()))
    freq = {}
    for i in lst:
        if i in freq:
            freq[i]+=1
        else:
            freq[i]=1
    res= []
    for i in freq:
        if freq[i] >= 2:
            res.append(i)
    if len(res)==0:
        return -1
    for i in res:
        print(i,end=" ")
duplicate()
```

```
 7 7 7 2 2 5
7 2
```

```
#  WEEK-5 PROGRAMMING ASSIGNMENT-3
# Q.
# A sequence of integers is given. Find all numbers that occur exactly two times.

# Write a function that:
# (a) Accepts a space-separated sequence of integers.
# (b) Returns all numbers that occur exactly two times.
# (c) The numbers should be returned in increasing order.
# (d) If no number occurs exactly two times, returns -1.

lst = list(map(int , input().split()))
freq = {}
for i in lst:
    if i in freq:
        freq[i]+=1
    else:
        freq[i]=1
res = []
for i in freq:
    if freq[i]==2:
        res.append(i)
if len(res)==0:
        print(-1)
res.sort()           # sorting to print increasing value
for i in res:
    print(i , end = " ")
```

```
 1 1 2 2 3 3
1 2 3
```

Start coding or generate with AI.

Start coding or generate with AI.

```
#  WEEK-6 PROGRAMMING ASSIGNMENT-1
# A cybersecurity company evaluates password strength by counting the number of alphabetic characters in a password. Due to memory constraints in a secure device, the counting

# Write a recursive function that:
# (a) Accepts a string s as input.
# (b) Returns an integer representing the number of alphabetic characters (A-Z and a-z) in the string.
# (c) Does not use loops.

def counter(s):
    if s=="":
        return 0
    if s[0].isalpha():
        return 1 + counter(s[1:])
    else:
        return counter(s[1:])

s=input()
print(counter(s))
```

```
 11abs
```
```
3
```

```
#  WEEK-6 PROGRAMMING ASSIGNMENT-2
# Q.
# A warehouse records the boxes currently stored in its inventory as a list of box IDs. Implement a program using recursion to determine the total number of boxes in the inve

#Write a program that:
# (a) Accepts space-separated integers as input, where each integer represents a box ID.
# (b) Uses recursion to count the number of boxes and displays the count as an integer.
# (c) Does not use loops or the len() function.

def counter(lst):
    if lst == []:
        return 0
    else:
        return 1 + counter(lst[1:])
lst = list(map(int , input().split()))
print(counter(lst))
```
```
 4 5 6 7 8 8
```
```
6
```

```
#  WEEK-6 PROGRAMMING ASSIGNMENT-3
# Q.
# A weather station records daily temperatures as a list of values. Implement a program using recursion to determine the highest temperature recorded.

# Write a program that:
# (a) Accepts space-separated integers as input, where each integer represents a recorded temperature.
# (b) Uses recursion to find and display the highest temperature as an integer.
# (c) Does not use loops or the max() function.

def find_max(temp):
    if len(temp)==1:
        return temp[0]
    high = find_max(temp[1:])
    if temp[0]>high:
        return temp[0]
    return high

temp = list(map(int, input().split()))
find_max(temp)
```
```
  9 8 3 6
```
```
9
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
#  WEEK-7 PROGRAMMING ASSIGNMENT-1
# Q.
# Tourism applications store information about attractions using dictionaries for quick retrieval.
# Write a program that accepts attraction information and an attraction name. The program should print the rating of the attraction if it exists; otherwise, print -1.

# Input:
# The first input line contains the number of attractions N.
# The next N lines contain an attraction name followed by its rating separated by a space. Please refer to public test cases for sample.
# The last line contains the name of the attraction to search.

# Output:
# Print the rating of the attraction if it exists; otherwise, print -1.

# Note: Each attraction name is followed by its rating, which is a decimal number with exactly one digit after the decimal point.

N = int(input())
res = {}
for i in range(N):
    attraction , rating = input().split()
    res[attraction] = rating
search = input()
if search in res:
    print(res[search])
else:
    print(-1)
```

```
 3
 Fort 5.0
 Beach 4.7
 Park 4.2
 Home
-1
```

```
#  WEEK-7 PROGRAMMING ASSIGNMENT-2
# A navigation system stores landmark names in a list.

# Write a program that accepts a list of landmark names and a landmark to search. The program should use linear search to find the landmark and print its index. If the landmar

# Input:

# The first input line contains the number of landmarks N.
# The next line contains N landmark names separated by spaces. All landmarks have single word in them.
# The last line contains the landmark name to search.
# Output:
# Print the index of the landmark if it is found; otherwise, print -1.

n = int(input())
landmarks = list(map(str , input().split()))
search = input()
if search in landmarks:
    print(landmarks.index(search))
else:
    print(-1)
```

```
 3
 Home Cinema Dome
 Dome
2
```

```
#  WEEK-7 PROGRAMMING ASSIGNMENT-3
# Q.

# A system receives a set of numbers and needs to identify the second largest number.

# Write a program that accepts a set of integers and prints the second largest number. Assume that all numbers are distinct.

# Input:
# A single line containing space-separated integers.

# Output:
# Print the second largest number.

lst = list(map(int , input().split()))
large = 0
secondlarge = 0
for i in lst:
    if large < i:
        secondlarge=large
        large = i
    elif i!=large and i > secondlarge:
        secondlarge=i
print(secondlarge)
```

```
 90 100 120 30 40 50
100
```

```
#  WEEK-8 PROGRAMMING ASSIGNMENT-1
# Modern elevators move either up one floor (+1) or down one floor (-1). Given a sequence of elevator movements, determine the final floor reached.

# Write a program that:
# (a) Accepts a single line containing space-separated integers, where each integer is either 1 or -1.
# (b) Assumes the elevator starts at floor 0.
# (c) Computes the final floor after applying all movements.
# (d) Prints the final floor.

lst = list(map(int, input().split()))
elevator = 0
for i in lst:
    if i == -1:
        elevator -= 1
    elif i == 1:
        elevator += 1
    else:
        pass
print(elevator)
```

```
 1 -1 1 -1 1 -1 1
1
```

Start coding or generate with AI.

Start coding or generate with AI.

```
# WEEK-8 PROGRAMMING ASSIGNMENT-2
# Q.
```

```
# An online quiz awards 10 points for every correct answer and deducts 2 points for every incorrect answer. Given the outcome of each question, determine the final score.

# Use the following representation:

# 1 → Correct answer
# 0 → Incorrect answer

# Write a program that:
# (a) Accepts a single line containing space-separated integers consisting of 0s and 1s.
# (b) Starts with a score of 0.
# (c) Computes the final score.
# (d) Prints the final score.

lst = list (map(int , input().split()))
score = 0
for i in lst:
    if i == 0:
        score -=2
    elif i == 1:
        score +=10
print(score)
```

```
 1 1 0 0 1 0 0
22
```

```
# WEEK-8 PROGRAMMING ASSIGNMENT-3
# Two employee IDs follow the same pattern if each character in the first ID always maps to the same character in the second ID, and different characters cannot map to the sa

# For example:

# foo and egg follow the same pattern because:

# f → e
# o → g
# The second o also maps to g
# The mapping is consistent, so the answer is True.

# However, ab and aa do not follow the same pattern because a → a and b → a. Two different characters (a and b) map to the same character.

# Write a program that:
# (a) Accepts a single line containing two space-separated strings id1 and id2.
# (b) Checks whether the two strings follow the same pattern.
# (c) Prints True if they follow the same pattern; otherwise, prints False.

id1, id2 = input().split()

# If lengths are different, they can't match
if len(id1) != len(id2):
    print(False)
else:
    d1 = {}
    d2 = {}
    same_pattern = True
    for a, b in zip(id1, id2):
        if (a in d1 and d1[a] != b) or (b in d2 and d2[b] != a):
            same_pattern = False
            break
        d1[a] = b
```

```
            d2[b] = a
        print(same_pattern)
```

```
# Week 09 - Programming Assignment 01
# Q.
# A university records the courses enrolled in by students. The course names are given as space-separated values. Write a program that accepts the list of course names and a

# Input:
# First line: space-separated course names
# Second line: the course name to search for
# Output:

# A single integer representing the number of students enrolled in the given course.

lst = list(map(str, input().split()))
search = input()
count = 0
for i in lst:
    if search == i:
        count+=1
print(count)
```

```
 C Python C Python
 Python
2
```

```
# Week 09 - Programming Assignment 02

# An organization blocks access to certain websites during working hours. The browsing history is given as space-separated website names, followed by the blocked website. Wri
# Input:
# A single line containing space-separated website names. The last value represents the blocked website.

# Output:
# Print a single integer representing the number of websites in the browsing history that are not blocked.

# Example:

# Input:
# google facebook github facebook

# Here, google, facebook, github is the browsing history and facebook is the blocked website. Since facebook is a blocked site, only google and github are not blocked. Hence

# Output:
# 2

lst = list(map(str , input().split()))
blocked=lst[-1]
count = 0
for i in lst:
    if i != blocked:
        count+=1
print(count)
```

```
 google facebook github facebook
2
```

```
# Week 09 - Programming Assignment 03
# Q.
# A company has several project teams. The number of employees assigned to each project is given as space-separated values. Write a program to find the difference between the

# Input:
# A single line containing space-separated integers. Each integer represents the number of employees assigned to one project team.

# Output:
# Print a single integer representing the difference between the largest and smallest team sizes.

# Example:

# Input:
# 4 7 3 9 5

# The largest team has 9 employees and the smallest team has 3 employees. Therefore, the difference is 6.
# Output:
# 6

lst = list(map(int, input().split()))
if len(lst)==1:
    print(lst[0])
else:
    print(max(lst)-min(lst))
```

```
 4 7 3 9 5
6
```

```
# Week 10 - Programming Assignment 01
# Q.

# In a treasure hunt, explorers stand in a circle and pass a magical token. Every k-th explorer holding the token leaves the game. The process continues until only one explor

# Write a program that:

# Accepts two space-separated integers n (nSumber of explorers) and k (passing count).
# Simulates the elimination process.
# Prints the position (1-indexed) of the last remaining explorer.

n , k = map(int, input().split())
people = list(range(1,n+1))
i = 0
while len(people) > 1:
    i= (i + k - 1)%len(people)
    people.pop(i)
print(people[0])
```

```
 10 5
3
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

```
# Week 10 - Programming Assignment 02

# A weather station records temperatures on a scale from 0 to 255 for sensor calibration. For analysis, the readings are grouped into 8 equal ranges.

# Write a program that:

# Accepts a list of integers in the range 0–255.
# Maps each value into one of 8 groups: 0–31 → 0, 32–63 → 1, …, 224–255 → 7.
# Adds all the resulting group numbers and prints the total.

lst = list(map(int , input().split()))
count=0
for i in lst:
    count = count + (i//32)
print(count)
```

```
 31 50 80 110 150 190 223 255
28
```

```
# Week 10 - Programming Assignment 03
# Q.

# Two warehouses maintain lists of product codes represented as strings. During inventory matching, identical product codes appearing in both warehouses are removed one at a

# Write a program that:
# Accepts two space-separated strings.
# Removes common characters from both strings, one occurrence at a time.
# Prints the total number of remaining characters.

first , second = input().split()
a = list(first)
b = list(second)
for i in first:
    if i in second:
        a.remove(i)
        b.remove(i)
print(len(a)+len(b))
```

```
 robot motor
2
```

```
# Week 11 - Programming Assignment 01
# Q.

# Online shopping platforms often need to quickly identify how many reviews contain a specific word.

# Write a program that:

# Accepts space-separated words, where the first word is the search word and the remaining words represent the reviews.
# Counts how many of the remaining words match the search word.
# Prints the count as a single number.

lst = list(map(str , input().split()))
search = lst[0]
reviews = lst[1:]
count = 0
for i in reviews:
```

```
    if search == i:
        count += 1
print(count)
```

```
 apple apple apple apple apple app
4
```

```
# Week 11 - Programming Assignment 02
# Q.

# Hotel reservation systems calculate the number of days between check-in and check-out dates for billing purposes.

# Write a program that:

# Accepts two dates in YYYY-MM-DD format as space-separated input.
# Computes the absolute number of days between the two dates.
# Prints the result as a single number.

from datetime import date
a,b = input().split()       # user types two dates(for example: 2021-01-01 2022-01-01)
y1,m1,d1 = map(int , a.split("-"))  #splits it at the hyphens into a list of strings and convert to int
y2,m2,d2 = map(int, b.split("-"))   #splits it at the hyphens into a list of strings and convert to int
date1 = date(y1,m1,d1)              #creating a date object
date2 = date(y2,m2,d2)              #creating a date object
print((date2-date1).days)
```

```
 2021-01-01 2022-01-01
365
```

```
# Week 11 - Programming Assignment 03
# Q.

# Many payroll systems need to determine the number of Sundays in a month for attendance calculations.

# Write a program that:

# Accepts a year and month as space-separated input.
# Uses the calendar module to determine the number of Sundays in that month.
# Prints the count as a single number.

import calendar
year, month = map(int, input().split())
weeks = calendar.monthcalendar(year, month)
count = 0
for week in weeks:
    if week[6] != 0:
        count += 1
print(count, end="")
```

```
 2025 2
4
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
# Week 12 - Programming Assignment 01
# Q.

# A search engine assigns every webpage an initial PageRank of 100.

# Suppose Page A links to Page B. During one update, Page A gives its entire PageRank to Page B.

# Given the PageRank of two pages, where the first page links to the second page, calculate the PageRank of both pages after one update.

# Input
# A single line containing two space-separated integers:

# rankA rankB

# Output
# Print the new PageRank of Page A and Page B, separated by a space.

pageA , pageB = map(int , input().split())
pageB = pageA+pageB
pageA = 0
print(pageA,pageB)
```

```
 500 300
0 800
```

```
# Week 12 - Programming Assignment 02
# Q.

# Two webpages, Page A and Page B, each have a PageRank. Page A links to Page B, while Page B links to Page A.

# During one update, each page gives its entire PageRank to the page it links to.

# Given their initial PageRanks, find their PageRanks after one update.

# Input
# A single line containing two space-separated integers

# Output

# Print the new PageRanks of Page A and Page B, separated by a space.

a , b = map(int, input().split())
b , a = a , b
print(a,b)
```

```
 175 500
500 175
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
# Week 12 - Programming Assignment 03
# Q.

# Three webpages have initial PageRanks.

# Page A links to Page B and Page C.
# Page B links to Page C.
# Page C has no outgoing link and keeps its rank.
# When a page has multiple links, it divides its PageRank equally among those links.

# Given the initial PageRanks of A, B, and C, calculate their PageRanks after one update.

# Input: The input contains three space-separated integers representing the initial PageRanks of Page A, Page B, and Page C, respectively. For example, 100 200 50 means that P
# Output: The output should contain three space-separated values representing the new PageRanks of Page A, Page B, and Page C after one update. Page A gives its rank equally t

# Note: There are no test cases that might result in decimal point results. Use integer division.

a , b , c = map(int, input().split())
c = b + (a//2) + c
b = a//2
a=0
print(a,b,c,end="")
```

```
 80 120 200
0 40 360
```

Start coding or generate with AI.